

OutfitAI PRD (AI 产品设计文档)

Part 1: 产品概述 + 用户分析 + 产品定位

1. 产品概述

1.1 产品名称

OutfitAI

产品类型:

基于多模态 AI 的移动端个人智能穿搭助手 (AI Personal Stylist)

1.2 一句话定位

OutfitAI 是一个通过视觉理解、用户画像和智能推荐技术，帮助用户管理个人衣橱，并根据场景、天气和个人风格生成个性化穿搭方案的 AI 造型助手。

1.3 产品背景

随着服装消费水平提升，用户衣橱规模不断扩大，但用户在日常穿搭过程中仍存在明显问题：

- 衣服数量增加，但实际使用率降低
- 用户购买大量服饰，却无法有效组合
- 面对不同场景缺少专业搭配建议
- 用户希望提升个人形象，但传统造型服务成本较高

目前市场上的穿搭工具主要集中于：

- 穿搭内容社区
 - 用户通过浏览达人内容寻找灵感
- 电商推荐系统
 - 根据商品销量和用户画像推荐商品
- AI 聊天助手
 - 根据文本描述提供泛化建议

但这些方案存在共同问题：

- 不理解用户真实衣橱
- 不知道用户已有服装资产
- 无法持续学习用户个人审美
- 推荐结果缺少个性化

因此，OutfitAI 希望通过：

Computer Vision + LLM + Recommendation System + User Memory

建立一个真正了解用户的 AI 穿搭伙伴。

1.4 产品愿景

Vision

成为用户日常生活中的个人 AI Stylist。

让每个人拥有一个：

“了解我的衣服、理解我的审美、知道我的生活场景，并持续优化穿搭选择的 AI 造型师。”

1.5 核心价值

对用户：

1. 降低穿搭决策成本

解决：

“今天穿什么？”

用户只需要输入：

明天面试，上海15度，希望专业但不要太正式

AI 自动生成方案。

2. 提升衣橱利用率

帮助用户发现：

已有衣服的新组合。

例如：

用户拥有：

- 白衬衫
- 黑裤子
- 灰色针织衫

AI发现：

可以组合：

- 面试 Look
- 商务休闲 Look
- 周末约会 Look

3. 降低购物决策成本

购买前：

用户上传商品链接：

AI分析：

“这件外套是否适合你的衣橱？”

避免：

买回来无法搭配。

2. 用户分析

2.1 目标用户

核心用户

用户群体：

18–35岁年轻消费者

包括：

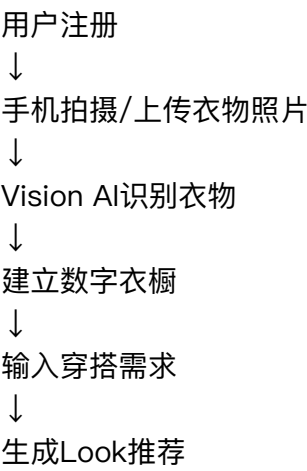
- 大学生
- 初入职场用户
- 城市白领
- 时尚兴趣用户

用户特征

共同特点：

特征	描述
有一定服装消费能力	每年购买多件衣服
重视个人形象	面试、社交、旅行需要
缺少专业搭配能力	不知道如何组合
高频使用手机	接受AI工具

用户流程设计



2.2 用户画像

Persona 1：职场新人 Emily

年龄：

23岁

身份：

互联网公司新人

场景：

每天通勤

痛点：

- 衣柜很多衣服
- 每天早上浪费时间选择
- 不知道什么适合职场

需求：

“帮我搭配专业但年轻的通勤穿搭。”

Persona 2：旅行用户 Alex

年龄：

28岁

场景：

北京旅行5天

痛点：

- 不知道带什么衣服
- 担心重复穿搭
- 不知道天气适配

需求：

“根据东京天气帮我规划5天旅行穿搭。”

Persona 3：时尚提升用户 Lily

年龄：

25岁

特点：

喜欢买衣服

痛点：

- 衣服很多但利用率低
- 想尝试新风格

需求：

“根据我的衣橱推荐新的搭配方式。”

2.3 用户痛点分析

痛点1：

每天不知道穿什么

用户行为：

打开衣柜 → 犹豫 → 固定穿几套

本质问题：

选择成本过高。

痛点2：

衣服很多但利用率低

用户拥有：

100件衣服

实际高频使用：

20件

原因：

用户不知道组合可能性。

痛点3：

特定场景缺少专业建议

例如：

- 面试
- 约会
- 商务会议
- 婚礼
- 旅行

用户需要：

场景化穿搭决策。

痛点4：

购物决策困难

用户购买：

新衣服

无法判断：

是否匹配已有衣橱。

导致：

冲动消费。

痛点5：

普通推荐缺少个人理解

传统推荐：

“热门穿搭”

但是用户真正需要：

“适合我的穿搭”。

2.4 用户使用场景

场景1：日常穿搭

输入：

今天20度，上班，希望简单干净

AI输出：

Look：

白衬衫

+

深蓝牛仔裤

+

风衣

并解释：

适合原因。

场景2：重要活动

输入：

下周面试，希望专业但不要太严肃

AI：

分析：

- 行业
- 场景
- 正式程度

生成：

Interview Outfit。

场景3：旅行规划

输入：

东京旅行5天，需要拍照好看的穿搭

AI：

结合：

- 天气API
- 行程
- 用户衣橱

生成：

Day1–Day5穿搭计划。

场景4：购物辅助

输入：

上传商品：

黑色皮衣

AI：

分析：

是否匹配：

用户已有：

- 白T
- 牛仔裤
- 靴子

输出：

匹配度：

85%

2.5 用户需求优先级

需求	优先级	原因
AI生成穿搭方案	P0	核心价值
上传衣物建立数字衣橱	P0	产品基础
根据天气/场景推荐	P0	提高准确率

解释推荐原因	P0	增强信任
收藏喜欢搭配	P1	建立反馈数据
购物搭配分析	P1	商业价值
长期个人风格学习	P1	提升留存
AI生成真人穿搭效果图	P2	增强体验
AR虚拟试穿	P2	技术复杂度高

3. 产品定位和竞品分析

3.1 产品定位

产品定位：

面向年轻用户的 AI 原生个人造型管理平台。

核心能力：

不是：

“告诉用户流行什么”

而是：

“根据用户已有资产和个人偏好，决定用户今天应该穿什么。”

3.2 竞品分析

竞品1：

小红书

优势：

- 穿搭内容丰富
- 用户社区活跃

不足：

- 内容推荐偏大众
- 不理解用户衣橱
- 无个人穿搭决策能力

OutfitAI机会：

从内容消费转向个人决策。

竞品2：

Stitch Fix

优势：

- 真人造型服务
- 个性化推荐

不足：

- 依赖人工
- 成本较高
- 实时交互弱

OutfitAI机会：

使用AI降低个人造型成本。

竞品3：

Whering

优势：

- 数字衣橱管理
- 衣服记录

不足：

- AI推荐能力有限
- 缺少复杂场景理解

OutfitAI机会：

从衣橱管理升级为主动决策。

3.3 核心竞争优势

Advantage 1:

Personal Closet Intelligence

不是推荐市场商品。

而是理解：

用户已有服装资产。

Advantage 2:

Multimodal AI

同时理解：

图片：

衣服属性

文本：

用户需求

外部数据：

天气、地点、活动

Advantage 3:

Long-term Memory

随着使用：

AI学习：

- 喜欢颜色
- 风格偏好
- 避免元素
- 常用场景

形成：

Personal Style Profile。

Advantage 4:

Explainable Recommendation

不是：

“推荐这套”

而是：

“为什么推荐这套”。

提升用户信任。

Part 2： 用户流程 + 核心功能设计

4. 用户流程设计

4.1 产品核心用户流程

用户注册



建立个人数字衣橱



上传衣服照片 / 导入购物链接



Vision AI识别衣物属性



生成个人衣橱数据库



用户提出穿搭需求

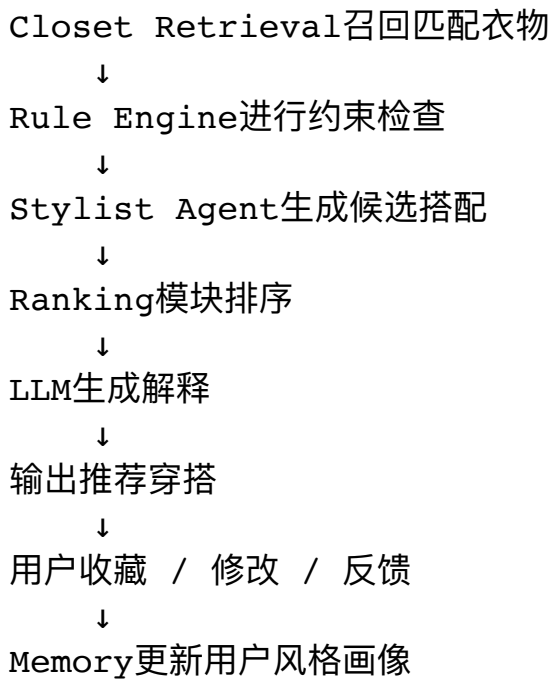


Requirement Agent解析需求



获取天气 / 场景 / 用户偏好数据





4.2 用户流程详细说明

Step 1: 用户建立数字衣橱

用户行为:

用户上传:

- 单件衣服照片
- 衣柜照片
- 电商购买链接

例如:

上传:

white-shirt.jpg

AI处理:

Vision Agent分析:

- 类别
- 颜色
- 材质

- 风格
- 季节
- 正式程度
- 保暖程度

输出：

```
{
  "category": "shirt",
  "color": [ "white" ],
  "material": "cotton",
  "style": [
    "minimal",
    "business casual"
  ],
  "season": [
    "spring",
    "autumn"
  ],
  "formality": 0.75,
  "warmth": 2
}
```

Step 2: 用户输入穿搭需求

用户输入：

自然语言：

明天上海15度面试，希望专业一点，但不要像银行员工。

系统不会直接推荐。

首先进入：

Requirement Agent。

Step 3: 需求理解

原始输入：

明天上海15度面试，希望专业但不要太严肃

转换:

```
{
  "occasion": "interview",
  "location": "Shanghai",
  "temperature": 15,
  "style_preference": "professional casual",
  "formality": 0.7,
  "avoid": [
    "too formal",
    "old-fashioned"
  ]
}
```

Step 4: 衣橱召回

系统查询:

用户衣橱数据库。

例如:

100件衣服。

通过:

Embedding Search

用户需求:

professional casual interview

转换Embedding。

匹配:

White shirt
Navy blazer
Black trousers
Leather shoes

Step 5: 规则检查

避免:

LLM犯错误。

例如:

天气:

35°C

规则:

temperature > 30

avoid:

wool coat
heavy jacket
boots

场景:

Interview

规则:

avoid:

shorts
slippers
graphic tshirt

Step 6: 生成推荐

Stylist Agent:

组合:

Look A:

白衬衫
+

深蓝西装外套
+
黑色直筒裤

Look B:

针织衫
+
西裤
+
乐福鞋

Step 7: 解释

LLM Explanation:

输出:

为什么推荐。

例如:

白衬衫和深蓝外套属于低风险商务配色，能够保持专业感，同时避免完整西装带来的距离感，更符合年轻职场面试场景。

5. 核心功能设计

功能1: AI数字衣橱

功能名称:

Personal Digital Closet

用户需求:

用户希望:

快速整理自己的衣服，并让AI了解已有服装资产。

功能描述：

用户上传服装图片，AI自动识别并生成结构化衣物信息。

支持：

P0

单件上传：

照片 → AI识别

P1

批量上传：

衣柜照片 → 自动拆分衣物

P1

购物链接导入：

淘宝 / 京东 / 小红书商品链接

P2

自动同步购物记录

输入：

图片：

shirt.jpg

或者：

商品链接：

<https://xxx.com/product/123>

AI处理逻辑：

流程：

Image Input

↓

Vision Model

↓

Clothing Attribute Extraction

↓

Structured Output

↓

Database Storage

使用：

- GPT-4o Vision
- Gemini Vision
- Qwen-VL

输出：

衣物卡片：

白色衬衫

Category:
Shirt

Style:
Minimal
Business Casual

Season:
Spring Autumn

Warmth:
2/5

Formality:
0.75

异常情况:

图片质量低

处理:

要求重新上传。

无法识别衣服

处理:

允许用户手动编辑。

MVP优先级:

P0

原因:

没有数字衣橱，后续推荐无法个性化。

功能2：AI穿搭推荐

功能名称:

AI Outfit Generator

用户需求:

用户希望根据:

- 场景
- 天气
- 风格

快速获得穿搭方案。

功能描述：

用户输入自然语言需求。

AI从用户衣橱中生成最佳搭配。

输入：

文本：

周五约会

20度

想要韩系简约风

系统额外获取：

Weather API

User Profile

Closet Database

AI处理逻辑：

Workflow：

User Query

↓

Requirement Agent

↓

Closet Retrieval Agent

↓

Rule Engine

↓

Stylist Agent

↓

Ranking

↓

LLM Explanation

输出：

穿搭方案：

Look 1：

灰色针织衫

+

黑色直筒裤

+

白色运动鞋

适合原因：

符合韩系极简风，
颜色低饱和，
适合20度天气。

异常情况：

用户衣服不足

例如：

用户没有西装。

解决：

推荐：

已有衣服最佳方案。

同时提示：

“如果购买，可以考虑xxx类型单品。”

MVP优先级：

P0

核心价值。

功能3：场景化穿搭规划

功能名称：

Scenario Outfit Planner

用户需求：

用户面对特殊场景不知道如何准备。

场景：

- 面试
- 旅行
- 婚礼
- 商务会议
- 约会

功能描述：

AI根据场景生成完整穿搭策略。

输入：

北京旅行5天

天气15-22度

想拍照

AI处理逻辑：

调用：

Requirement Agent

提取：

location

weather

activities

photo_style

duration

调用：

Weather API

生成：

Day 1-Day 5 Outfit Plan。

输出：

Day1

浅色风衣
+

白T
+
牛仔裤

Day2

针织衫
+
半裙

异常：

旅行天气变化。

解决：

实时更新推荐。

MVP优先级：

P1

原因：

增加用户使用频率。

功能4：购物搭配助手

功能名称：

Shopping Compatibility Checker

用户需求：

用户购买前想知道：

“这件衣服适不适合我？”

功能描述：

用户上传商品图片。

AI结合用户衣橱分析。

输入：

商品：

black leather jacket

用户衣橱：

white T-shirt

blue jeans

boots

AI处理：

Vision：

分析商品。

Retrieval：

查找匹配衣服。

Ranking：

计算搭配数量。

输出：

推荐指数：

87%

可以搭配：

12套

适合风格：

Minimal
Street

MVP优先级：

P1

原因：

商业化潜力高。

功能5：个人风格记忆

功能名称：

AI Style Memory

用户需求：

用户希望AI越来越懂自己。

功能描述：

记录：

用户行为：

- 收藏
- 删除
- 修改
- 喜欢风格

形成：

Personal Style Profile。

输入：

用户反馈：

不喜欢这套

颜色太亮

AI处理：

更新：

User Preference Vector

例如：

之前：

`minimal:0.5`

之后：

`minimal:0.75`

输出：

未来推荐优化。

MVP优先级：

P1

6. MVP功能范围总结

功能	P0/P1/ P2	原因
上传衣服建立数字衣橱	P0	核心数据基础
AI衣物识别	P0	减少用户输入成本
自然语言穿搭需求	P0	核心交互
AI穿搭推荐	P0	核心价值
推荐解释	P0	提高信任
收藏反馈	P1	形成数据闭环
旅行穿搭规划	P1	提高使用频率
购物搭配分析	P1	商业化入口
个人风格模型	P1	提升长期价值
AI生成试穿图	P2	增强体验
AR虚拟试穿	P2	技术复杂度高

Part 3: AI 系统架构 + Agent / Workflow 设计

6. AI 系统架构设计

6.1 总体架构目标

OutfitAI 不是一个简单的 LLM Chatbot，而是一个：

多模态理解 + RAG检索 + Agent Workflow + 推荐系统 + 用户长期记忆

组成的 AI Native 产品。

核心设计原则：

1. LLM 不负责所有决策

避免：

- 幻觉推荐
- 不符合天气

- 不符合场景

采用：

LLM + 数据库 + Rule Engine + Retrieval。

2. Agent负责复杂任务拆解

例如：

用户：

东京旅行5天，想拍照好看，需要带什么衣服？

系统拆解：

理解需求

↓

获取天气

↓

查询用户衣橱

↓

生成搭配

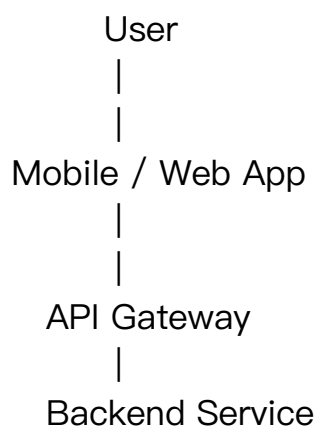
↓

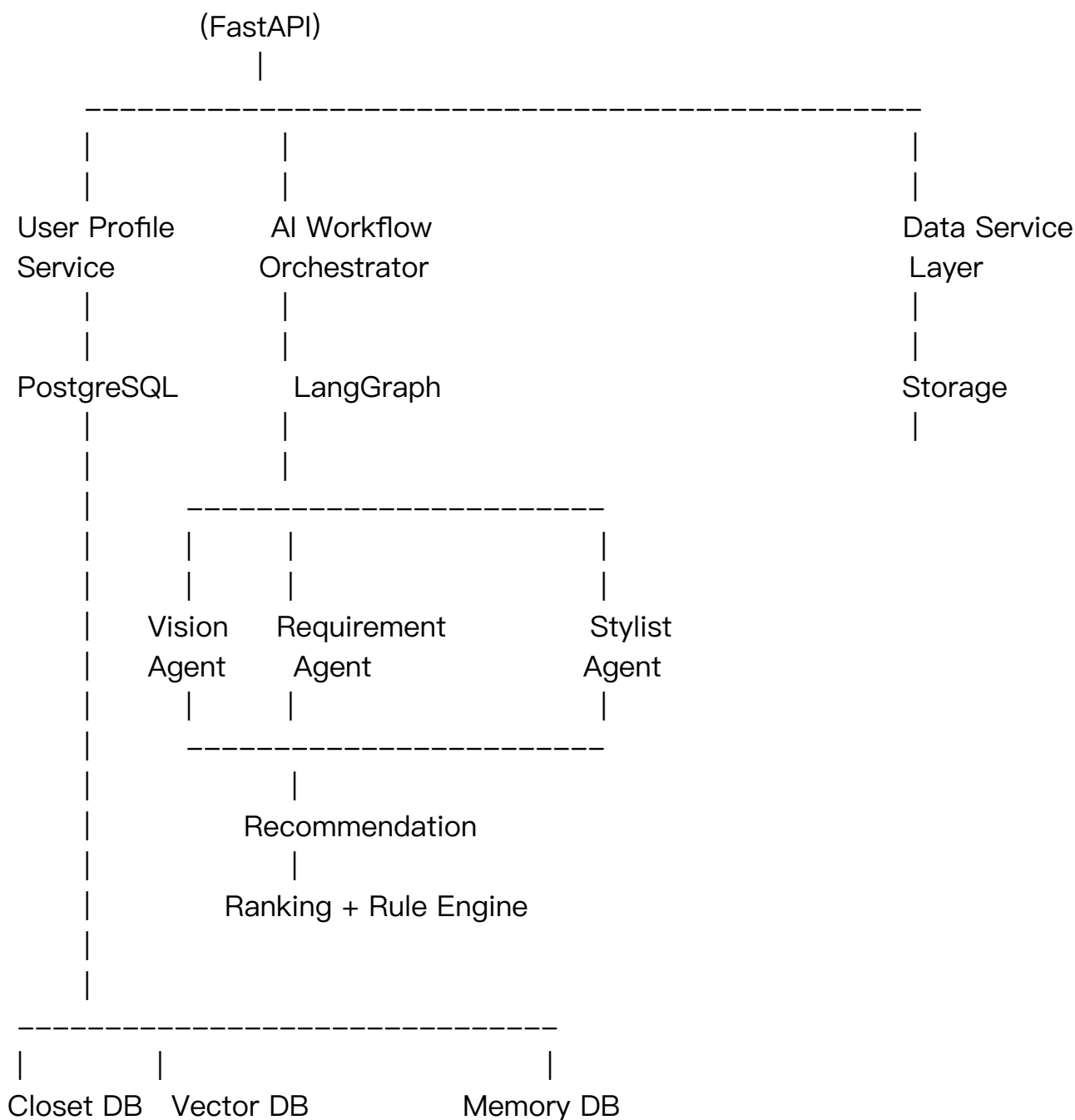
检查合理性

↓

解释结果

6.2 系统整体架构





6.3 技术分层设计

Layer 1: Frontend

技术：

推荐：

- React Native（移动端）

- Next.js (Web)

MVP:

Streamlit / React 均可。

负责：

用户交互：

- 上传衣服照片
- 输入需求
- 查看穿搭方案
- 收藏反馈

Layer 2: Backend

技术：

FastAPI

负责：

- 用户认证
- API管理
- Workflow调用
- 数据处理

接口示例：

上传衣服

POST:

/closet/upload

输入：

```
{
  "user_id":123,
  "image_url":"shirt.jpg"
}
```

获取穿搭推荐

POST:

/outfit/generate

输入:

```
{  
  "user_id":123,  
  
  "query":  
  "明天面试, 15度, 希望专业但年轻"  
}
```

Layer 3: LLM Layer

作用:

负责:

1. 自然语言理解

例如:

用户:

不想看起来像销售

转换:

```
{  
  "avoid_style":  
  [  
    "too formal",  
    "old fashioned"
```

```
]
}
```

2. 生成解释

例如：

解释：

为什么选择：

白衬衫+蓝西装。

3. Agent推理

例如：

决定下一步调用：

天气API？

还是衣橱搜索？

推荐模型：

商业：

- GPT-4o
- Claude
- Gemini

国内：

- Qwen
- DeepSeek

Layer 4: Agent Layer

为什么需要 Agent？

传统流程：

用户输入



一个Prompt



LLM回答

问题：

无法控制：

- 数据来源
- 工具调用
- 任务步骤

Agent方式：

用户需求



任务拆解



调用工具



综合结果



输出

更适合复杂穿搭任务。

Layer 5: RAG Layer

为什么需要RAG?

OutfitAI需要大量外部知识:

例如:

- 色彩搭配规则
- 职业穿搭规范
- 季节穿衣建议

这些不应该全部放Prompt。

RAG知识库:

包含:

Style Knowledge

例如:

Business casual:

推荐:

shirt

blazer

trousers

避免:

shorts

slippers

Color Matching

例如:

Navy:

compatible:

white

grey

black

Occasion Rules

Interview:

formality ≥ 0.7

流程：

Knowledge Base



Embedding Model



Vector Database



Similarity Search



Agent Context

Layer 6: Database Layer

关系数据库

PostgreSQL

保存：

用户信息

衣物信息

反馈记录

Object Storage

例如：

- AWS S3
- Supabase Storage

保存：

图片。

Vector Database

推荐：

- pgvector
- Milvus
- FAISS

保存：

衣物embedding。

Layer 7: External API Layer

外部数据：

Weather API

作用：

天气适配。

例如：

输入：

Shanghai tomorrow

输出：

temperature:

15

humidity:

70%

weather:

cloudy

Shopping API

未来：

商品分析。

7. Agent / Workflow设计

7.1 Workflow总体设计

Outfit Recommendation Workflow:

User Request

↓

Requirement Agent



Context Collection



Closet Retrieval Agent



Candidate Generation



Rule Engine Validation



Ranking Model



Stylist Agent



Explanation Agent



User Feedback



Memory Update

7.2 Agent划分原则

使用Agent：

适合：

- 非结构化输入
- 需要推理
- 需要调用工具

使用传统代码：

适合：

- 固定规则
- 数据查询
- 计算

8. 核心Agent设计

Agent 1: Vision Agent

职责：

理解用户上传衣服。

输入：

图片：

shirt.jpg

处理：

Vision Model

识别：

- 类别
- 颜色
- 风格
- 材质
- 季节
- 正式程度

输出：

```
{  
  "category": "shirt",  
  
  "attributes":  
  {  
    "color": "white",  
    "style":  
    [  
      "minimal",  
      "business"  
    ],  
  
    "warmth": 2,  
  
    "formality": 0.8  
  }  
}
```

调用模型：

GPT-4o Vision

Qwen-VL

工具：

Image Processor

Agent 2: Requirement Agent

职责：

理解用户穿搭需求。

输入：

明天面试，希望专业但不要太严肃

输出：

```
{  
  "occasion":  
    "interview",  
  
  "style":  
    "professional casual",  
  
  "formality":0.7,  
  
  "avoid":  
    [  
      "too formal"  
    ]  
}
```

调用：

LLM

工具：

Agent 3: Closet Retrieval Agent

职责：

从用户衣橱寻找候选衣服。

输入：

Requirement JSON

查询：

Vector Database

例如：

Query:

professional interview outfit

匹配：

white shirt

navy blazer

black trousers

输出：

Top-K衣物。

技术：

Embedding：

BAAI/bge

OpenAI Embedding

Vector：

Milvus

pgvector

Agent 4： Stylist Agent

职责：

组合穿搭。

输入：

候选衣物：

shirt

pants

shoes

输出：

多个Look。

例如：

```
{  
  "outfits":[
```

```
{  
  "top":  
    "white shirt",  
  
  "bottom":  
    "black pants",  
  
  "shoes":  
    "leather shoes"  
}  
  
]
```

Agent 5: Feedback Agent

职责：

学习用户偏好。

输入：

用户：

“不喜欢这套”

分析：

原因：

颜色？

风格？

正式程度？

更新：

User Memory。

9. Rule Engine设计

为什么需要Rule Engine?

LLM具有创造能力。

但是：

不能保证约束。

例如：

用户：

夏天35度。

LLM可能：

推荐羊毛大衣。

所以：

需要硬规则。

规则示例

天气规则：

```
{
  "temperature_gt_30":
  {
    "avoid":
    [
      "wool",
      "heavy jacket"
    ]
  }
}
```

场景规则：

```
{
  "interview":
  {
    "minimum_formality":
    0.7,

    "avoid":
    [
      "shorts"
    ]
  }
}
```

10. Recommendation Ranking设计

MVP阶段：

不训练模型。

采用：

规则评分 + LLM Judge

评分：

Final Score =

30% Occasion Match

+

25% Style Match

+

20% Weather Match

+

15% Color Harmony

+

10% User Preference

输入：

```
{  
  "outfit":  
  [  
    "white shirt",  
    "black pants"  
  ]  
}
```

输出：

```
{  
  "score":0.91  
}
```

11. Memory Layer设计

目标：

让AI越来越懂用户。

保存：

用户风格：

```
{
  "preferred_style":

  [
    "minimal",
    "korean"
  ]
}
```

避免：

```
{
  "avoid":

  [
    "bright colors",
    "tight pants"
  ]
}
```

历史行为：

```
{
  "favorite_outfits":

  [
    001,
    005
  ]
}
```

更新：

User Feedback

↓

Preference Extraction

↓

Memory Update

↓

Future Recommendation

Part 4: 数据设计 + 技术方案 + MVP规划 + 指标 + 风险 + Roadmap

8. 数据设计

OutfitAI 的核心数据资产不是衣服图片，而是：

用户衣橱数据 + 用户风格偏好 + 穿搭反馈数据 + 场景知识数据

这些数据会随着用户使用不断增强 AI 个性化能力。

8.1 User Request 数据结构

用户自然语言需求需要经过 Requirement Agent 转换成结构化数据。

User Request JSON

```
{  
  "user_id": "12345",  
  
  "request_text":  
    "明天上海面试，15度，希望专业但不要太严肃",
```

```
"occasion":
"interview",

"location":
{
  "city":"Shanghai",
  "country":"China"
},

"weather":
{
  "temperature":15,
  "weather":"cloudy"
},

"style_preference":
[
  "professional",
  "minimal"
],

"formality_level":
0.7,

"avoid_style":
[
  "too formal",
  "old fashioned"
],

"color_preference":
[
  "neutral"
]
}
```

8.2 Closet Database Schema

表1： User

用户基础信息。

字段	类型	说明
user_id	string	用户ID
gender	string	性别
age	int	年龄
location	string	地区
created_time	datetime	注册时间

表2： Clothing Item

核心衣物表。

Clothing_Item

字段	类型	说明
item_id	string	衣物ID
user_id	string	所属用户
image_url	string	图片地址
category	string	类别
color	array	颜色
material	string	材质
style	array	风格
season	array	适合季节
warmth_score	float	保暖程度
formality_score	float	正式程度
embedding	vector	语义向量

created_time	datetime	创建时间
--------------	----------	------

示例：

```
{
  "item_id": "001",

  "category":
  "shirt",

  "color":
  [
    "white"
  ],

  "style":
  [
    "minimal",
    "business"
  ],

  "formality_score":
  0.8
}
```

表3： Outfit History

记录生成过的穿搭。

字段	类型	说明
outfit_id	string	搭配ID
user_id	string	用户
items	array	衣物组合
occasion	string	场景

score	float	推荐分数
liked	boolean	是否喜欢
created_time	datetime	时间

表4： User Preference Memory

用户长期偏好。

```
{
  "user_id": "123",

  "preferred_style":
  [
    "minimal",
    "korean"
  ],

  "preferred_colors":
  [
    "black",
    "white",
    "grey"
  ],

  "avoid_items":
  [
    "bright color",
    "tight pants"
  ],

  "favorite_outfits":
  [
    001,
    005
  ]
}
```

8.3 Knowledge Base Schema

用于 RAG。

Style Knowledge

```
{  
  "type":  
  "style_rule",  
  
  "title":  
  "Business Casual",  
  
  "content":  
  
  "Business casual usually includes shirts,  
  blazers and trousers.  
  Avoid shorts and slippers."  
}
```

Occasion Rule

```
{  
  "type":  
  "occasion_rule",  
  
  "occasion":  
  "interview",  
  
  "rules":  
  {  
    "minimum_formality":  
    0.7,  

```

```
"avoid":  
[  
  "graphic tshirt"  
]  
}  
}
```

Color Matching Knowledge

```
{  
  "type":  
    "color_rule",  
  
  "base_color":  
    "navy",  
  
  "compatible":  
    [  
      "white",  
      "grey",  
      "black"  
    ]  
}
```

8.4 Agent Output Schema

所有 Agent 输出统一 JSON。

原因：

方便：

- 调试
- 监控
- 前后端交互
- 后续模型优化

Requirement Agent Output

```
{  
  "occasion":  
    "travel",  
  
  "location":  
    "Tokyo",  
  
  "duration":  
    5,  
  
  "style":  
    "photo friendly",  
  
  "weather_required":  
    true  
}
```

Vision Agent Output

```
{  
  "item_id":  
    001,  
  
  "category":  
    "jacket",  
  
  "attributes":  
    {  
      "color":  
        "black",  
  
      "style":  
        [  
          "street"
```

```
],
```

```
"warmth":
```

```
4
```

```
}
```

```
}
```

Stylist Agent Output

```
{
```

```
"recommendations":
```

```
[
```

```
{
```

```
"items":
```

```
[
```

```
"white shirt",
```

```
"black pants",
```

```
"sneakers"
```

```
],
```

```
"reason":
```

```
"minimal clean style"
```

```
}
```

```
]
```

```
}
```

9. 技术方案

9.1 前端

MVP:

React + Next.js

原因：

- 快速开发
- 生态成熟
- 适合AI Web App

未来：

React Native

开发移动端。

9.2 Backend

FastAPI

原因：

- Python AI生态完善
- 易调用模型API
- 异步性能好

负责：

- 用户管理
- API
- Agent Workflow
- 数据处理

9.3 Workflow Orchestration

推荐：

LangGraph

原因：

OutfitAI流程具有：

- 多步骤
- 状态保存
- Agent调用

- 条件分支

例如：

Requirement Agent

if weather_required:

 call Weather API

if closet_empty:

 ask user upload clothes

continue:

Stylist Agent

9.4 LLM选择

文本模型

海外：

- GPT-4o
- Claude

国内：

- DeepSeek
- Qwen

负责：

- 用户需求理解
- 推荐解释
- 风格分析

Vision模型

推荐：

- GPT-4o Vision
- Qwen-VL
- Gemini Vision

负责：

图片理解。

9.5 Embedding模型

用途：

衣服语义搜索。

例如：

衣服：

white shirt minimal business

用户需求：

interview professional outfit

转换向量。

推荐：

- OpenAI Embedding
- BGE Embedding

9.6 Vector Database

选择：

MVP：

pgvector

原因：

- 和PostgreSQL统一
- 成本低
- 开发简单

规模扩大：

Milvus

9.7 数据库

PostgreSQL

保存：

- 用户
- 衣物
- 历史推荐
- Feedback

Object Storage

例如：

- AWS S3
- Supabase Storage

保存：

图片。

9.8 External API

Weather API

用途：

天气适配。

输入：

城市+日期

输出：

温度、天气、湿度。

Shopping API（未来）

用途：

商品分析。

10. MVP版本规划

P0：核心MVP版本

目标：

验证：

“用户是否愿意让AI帮自己决定穿什么”

周期：

8-12周

功能：

1. 用户上传衣服

支持：

单件照片上传。

2. AI衣物识别

输出：

结构化衣物信息。

3. AI穿搭推荐

输入：

场景+天气+风格

输出：

3套搭配。

4. 推荐解释

告诉用户：

为什么这样搭。

5. 收藏功能

记录：

喜欢/不喜欢。

为什么P0：

因为形成最小闭环：

衣服数据

↓

AI理解

↓

推荐

↓

用户反馈

↓

优化

P1： 体验增强

周期：

3-6个月

功能：

1. AI旅行穿搭规划

例如：

东京5天旅行。

输出：

Day1-Day5。

2. 购物搭配助手

上传商品：

判断：

是否适合已有衣橱。

3. 用户风格Memory

AI学习：

用户偏好。

4. 自动衣柜整理

批量图片识别。

P2：商业化和长期能力

1. AI虚拟试穿

技术：

- Diffusion Model
- Human Segmentation
- Pose Control

2. AR试穿

使用：

Camera API

3. 商业推荐

用户授权情况下：

推荐购买：

- 缺少单品
- 风格补充

商业模式：

佣金。

11. 风险和挑战

11.1 AI幻觉风险

问题：

LLM可能推荐：

不存在衣服。

解决：

所有推荐必须来自：

Closet Database。

LLM只能组合已有item。

11.2 推荐准确性

问题：

AI理解错误：

“正式但不要严肃”

解决：

Requirement Agent结构化。

加入：

用户反馈。

11.3 数据不足

问题：

新用户没有衣橱。

解决：

冷启动：

- 推荐基础模板
- 允许手动选择风格
- 渐进式建立衣橱

11.4 实时信息问题

天气变化。

解决：

Weather API实时调用。

11.5 成本问题

Vision调用成本高。

优化：

1.
图片压缩。
2.
首次识别保存结果。
3.
缓存Embedding。

11.6 用户上传成本

最大用户阻力。

解决：

渐进式Onboarding

第一次：

上传10件常穿衣服。

之后：

慢慢增加。

12. 产品指标

12.1 用户指标

激活率

定义：

完成首次衣橱建立并生成第一套穿搭比例。

目标：

60%

留存率

7日留存：

衡量用户是否每天使用。

使用频率

指标：

每周生成穿搭次数。

12.2 AI指标

1. Recommendation Acceptance Rate

用户接受推荐比例。

公式：

Accepted Outfit

/

Generated Outfit

2. Precision@K

例如：

推荐Top3：

用户喜欢几个。

Relevant Items in Top K

/

K

3. Recall@K

用户喜欢的衣服：

被推荐覆盖多少。

4. 用户满意度

评分：

1-5。

12.3 业务指标

转化率

购物助手：

推荐商品点击率。

收藏率

衡量：

推荐价值。

13. 产品Roadmap

第一阶段

目标：

完成AI穿搭MVP。

上线：

✓ 数字衣橱

✓ Vision识别

✓ AI推荐

✓ 解释

✓ 反馈

验证：

用户是否愿意使用。

第二阶段

目标：

增强个性化。

新增：

✓ 旅行规划

✓ 购物助手

✓ Memory系统

✓ 风格画像

形成：

Personal AI Stylist。

第三阶段

目标：

AI Fashion Platform

能力：

- AR试穿
- AI造型生成
- 虚拟模特
- 社交分享